



QUIZ 3 : ENTRETIEN DEVELOPPEUR NEXORA - CORRIGÉ

Évaluation Technique - Fullstack Junior

Répartition des questions

Domaine	Nombre de questions
Langages & Concepts (JS / TS / Python)	6
Architecture Frontend (Composants, État, Cycles)	7
Architecture Backend & APIs (Routage, DTO, ORM)	8
Bases de données & Persistance (SQL, Index)	6
Sécurité & DevOps (Docker, Git, CORS, Hashing)	8
Performance, IA & Méthodologies de coding (Claude Code)	10
Cas Pratiques	5
Total	50

Questionnaire à Choix Multiples (QCM)

1. Différence entre interface et type.

- ☐ A. interface est déprécié en TypeScript
- ☐ B. type ne fonctionne qu'avec les objets
- ☒ C. interface peut être étendue/fusionnée, type supporte les unions et intersections
- ☐ D. Identiques en tout point, aucune différence

2. Qu'est-ce qu'une Promise ?

- ☐ A. Un type de tableau dynamique
- ☐ B. Un décorateur TypeScript
- ☐ C. Une fonction qui s'exécute de façon synchrone
- ☒ D. Objet représentant le résultat éventuel d'une opération asynchrone (résolu, rejeté, en attente)

3. À quoi sert la méthode map() en JavaScript ?

- ☒ **A. À créer un nouveau tableau en appliquant une fonction sur chaque élément du tableau d'origine**
- ☐ B. À vérifier si au moins un élément respecte une condition
- ☐ C. À supprimer le premier élément d'un tableau
- ☐ D. À trier les éléments d'un tableau par ordre alphabétique

4. À quoi sert let par rapport à var ?

- ☐ A. let est uniquement utilisable dans les fonctions asynchrones
- ☒ **B. let a un scope de bloc et évite le hoisting indésirable, var a un scope de fonction**
- ☐ C. Identiques, aucune différence
- ☐ D. var empêche la redéclaration de variable

5. Qu'est-ce qu'un gestionnaire de paquets (comme npm ou pip) ?

- ☐ A. Un moteur de base de données relationnelle
- ☒ **B. Un outil permettant d'installer, de partager et de gérer les dépendances de projet**
- ☐ C. Un outil de déploiement dans le cloud
- ☐ D. Un framework backend pour créer des serveurs

6. Rôle d'un Controller.

- ☐ A. Valider la structure de la table SQL
- ☐ B. Gérer la connexion directe à la base de données
- ☐ C. Compiler le code TypeScript en JavaScript
- ☒ **D. Recevoir les requêtes HTTP entrantes et retourner les réponses adaptées au client**

7. Qu'est-ce qu'un DTO (Data Transfer Object) ?

- ☒ **A. Un objet qui définit la structure et le format des données envoyées dans une requête HTTP**
- ☐ B. Un type de composant graphique
- ☐ C. Un outil de migration de base de données
- ☐ D. Un middleware de chiffrement des mots de passe

8. À quoi sert un service dans une application backend ?

- ☒ **A. À isoler la logique métier et les calculs pour les réutiliser ailleurs**
- ☐ B. À gérer uniquement le rendu visuel des pages
- ☐ C. À remplacer l'utilisation de la base de données
- ☐ D. À configurer les ports réseau du serveur

9. Expliquez l'injection de dépendances.

- ☐ A. Un mécanisme pour importer des fichiers CSS
- ☐ B. Une technique pour insérer des données de test en base de données
- ☐ C. Une faille de sécurité liée aux formulaires
- ☒ **D. Pattern où les dépendances d'une classe sont fournies par un conteneur externe plutôt qu'instanciées manuellement**

10. Dans un serveur, comment lit-on généralement le contenu d'un fichier de manière non bloquante (asynchrone) ?

- ☐ A. En utilisant une boucle d'attente active (busy-waiting) synchrone
- ☐ B. En important le fichier directement au runtime dans le code source
- ☐ C. Les serveurs modernes ne permettent pas de lire des fichiers locaux
- ☒ D. En utilisant des appels asynchrones avec des promesses (Promises), des callbacks ou des fonctions `async/await`

11. À quoi sert le constructeur (comme `__init__` en Python ou `constructor` en JS) dans une classe ?

- ☐ A. À détruire une instance de classe libérée de la mémoire
- ☐ B. À importer un module externe dans la classe
- ☐ C. À déclarer des variables globales à l'application
- ☒ D. À initialiser les attributs d'un nouvel objet lors de sa création

12. Différence entre un tableau/liste et un objet/dictionnaire/hashmap.

- ☐ A. Le tableau utilise des clés textuelles, le dictionnaire utilise des index numériques
- ☐ B. Les tableaux sont immuables, les dictionnaires sont mutables
- ☒ C. Le tableau est une collection ordonnée indexée numériquement, le dictionnaire stocke des paires clé-valeur
- ☐ D. Il n'y a aucune différence sémantique ou d'accès

13. Qu'est-ce qu'une Migration ?

- ☐ A. Le transfert d'une application d'un serveur local vers le cloud
- ☐ B. Un changement de version du langage de programmation
- ☒ C. Un fichier de code décrivant les changements à appliquer à la structure de la base de données
- ☐ D. Une requête de synchronisation entre deux API

14. Qu'est-ce que l'ORM ?

- ☐ A. Un protocole de communication en temps réel
- ☐ B. Un système d'authentification par jeton
- ☒ C. Une couche d'abstraction permettant d'interagir avec la base de données en utilisant des objets du langage de programmation
- ☐ D. Un outil de minification du code source

15. Quel est le rôle principal d'un serveur de développement local (comme `runserver`, `npm run dev`, etc.) ?

- ☐ A. À appliquer automatiquement les migrations de base de données
- ☐ B. À générer la documentation du projet
- ☒ C. À lancer et exécuter l'application localement avec rechargement à chaud (hot reload) pour faciliter le développement
- ☐ D. À compiler définitivement le code pour la production

16. À quoi sert un sérialiseur (Serializer) ou un DTO dans une API ?

- ☐ A. À trier les résultats d'une recherche
- ☐ B. À sécuriser les routes d'une API
- ☐ C. À compresser les fichiers d'images sur le serveur
- ☒ D. À convertir des objets de données complexes (comme des objets de base de données) en un format simple standardisé (comme JSON) et inversement

17. Dans un framework frontend moderne, à quoi sert la gestion de l'état local (state) d'un composant ?

- ☐ A. À configurer les variables d'environnement globales de l'application
- ☐ B. À effectuer des requêtes HTTP asynchrones
- ☐ C. À recharger la page du navigateur à chaque interaction
- ☒ D. À stocker des données dynamiques propres au composant, dont la mise à jour déclenche automatiquement un re-rendu de l'affichage

18. Qu'est-ce que le JSX ou les syntaxes de templating déclaratives similaires ?

- ☐ A. Un outil de test unitaire pour les applications mobiles
- ☐ B. Une extension de base de données NoSQL
- ☒ C. Une extension de syntaxe permettant d'écrire la structure visuelle (similaire à du HTML) directement dans le code logique (JS/TS)
- ☐ D. Un protocole de transfert de fichiers sécurisé

19. Qu'est-ce que les Props dans les architectures de composants modernes ?

- ☐ A. Des variables d'état internes et modifiables par le composant lui-même
- ☐ B. Des outils pour styliser les éléments graphiques
- ☒ C. Des données passées d'un composant parent à un composant enfant, généralement en lecture seule
- ☐ D. Des fonctions de configuration globale du serveur

20. Quel est le rôle principal d'un composant en développement d'interface ?

- ☐ A. Stocker l'intégralité de la base de données de l'application
- ☒ B. Encapsuler une portion de l'interface utilisateur et sa logique pour la rendre réutilisable
- ☐ C. Gérer l'authentification côté serveur
- ☐ D. Optimiser la bande passante du réseau

21. Différence majeure entre une application web responsive et une application mobile native.

- ☒ A. L'application responsive s'exécute dans un navigateur web, l'application native est installée directement sur le système de l'appareil
- ☐ B. Les applications natives ne peuvent pas consommer d'API REST
- ☐ C. L'application responsive ne fonctionne pas sans connexion internet
- ☐ D. Une application responsive est obligatoirement écrite en Python

22. Qu'est-ce que le re-rendu (re-render) d'un composant ?

- ☐ A. Le rafraîchissement manuel de la page par l'utilisateur
- ☐ B. Une erreur fatale qui fait planter l'application
- ☐ C. La compilation du code avant la mise en production
- ☒ D. Le processus par lequel un composant met à jour son affichage suite à un changement d'état ou de props

23. Qu'est-ce qu'une clé primaire ?

- ☐ A. Une clé de chiffrement pour sécuriser les tables
- ☐ B. La première colonne créée dans une base de données
- ☐ C. Un index de recherche textuelle

[X] D. Une colonne qui identifie de manière unique chaque ligne d'une table SQL

24. Différence entre JOIN et LEFT JOIN.

☐ A. Ce sont des synonymes exacts utilisables indifféremment

☐ B. LEFT JOIN efface la table de droite en cas d'absence de correspondance

[X] C. JOIN retourne uniquement les lignes ayant une correspondance dans les deux tables, LEFT JOIN retourne toutes les lignes de la table de gauche plus les correspondances

☐ D. JOIN est utilisé pour les nombres, LEFT JOIN pour les textes

25. À quoi sert un index en base de données ?

[X] A. À accélérer la vitesse de recherche des requêtes sur les colonnes indexées

☐ B. À lier deux bases de données distinctes

☐ C. À trier obligatoirement les résultats par ordre alphabétique

☐ D. À empêcher l'insertion de valeurs négatives

26. Qu'est-ce qu'une clé étrangère (Foreign Key) ?

☐ A. Une clé d'accès pour un utilisateur externe à l'entreprise

☐ B. Une table contenant des traductions

[X] C. Une colonne qui référence la clé primaire d'une autre table pour établir une relation entre elles

☐ D. Un mot de passe temporaire généré par l'API

27. Qu'est-ce qu'une base de données relationnelle ?

[X] A. Un système où les données sont organisées en tables avec des relations définies entre elles

☐ B. Une base de données utilisable uniquement en local

☐ C. Une base de données stockant les données sous forme de fichiers texte brut

☐ D. Un réseau social de stockage de données

28. Qu'est-ce qu'un JWT (JSON Web Token) ?

☐ A. Un outil de compression de fichiers JSON

[X] B. Un jeton sécurisé utilisé pour authentifier un utilisateur de manière sécurisée et stateless après sa connexion

☐ C. Un script de configuration pour les serveurs web

☐ D. Une extension de fichier pour les bases de données relationnelles

29. Qu'est-ce qu'une injection SQL ?

☐ A. Une méthode d'optimisation pour insérer des données plus rapidement

☐ B. Une erreur de syntaxe lors de l'écriture d'une jointure

[X] C. Une faille de sécurité où un attaquant introduit du code SQL malveillant dans les champs de saisie pour manipuler la base de données

☐ D. Une commande permettant de sauvegarder une base de données

30. Pourquoi est-il important de masquer les mots de passe en base de données (hashing) ?

☐ A. Pour accélérer le processus de connexion de l'utilisateur

☐ B. Pour réduire l'espace de stockage sur le disque du serveur

[X] C. Pour éviter qu'un accès non autorisé à la base de données ne révèle les mots de passe en clair

☐ D. C'est une contrainte imposée par les navigateurs web

31. À quoi sert Docker ?

☐ A. À concevoir des maquettes d'interfaces graphiques

☒ B. À emballer une application et ses dépendances dans un conteneur isolé pour garantir son fonctionnement sur n'importe quelle machine

☐ C. À héberger des sites web gratuitement en ligne

☐ D. À gérer les versions du code source en équipe

32. Qu'est-ce que Git ?

☐ A. Un service d'hébergement de bases de données cloud

☐ B. Un framework de développement frontend

☐ C. Un environnement d'exécution pour le code JavaScript

☒ D. Un système de contrôle de version décentralisé pour suivre les modifications du code source

33. À quoi sert un fichier .env ?

☐ A. À stocker le code de l'interface utilisateur

☒ B. À conserver les variables de configuration et les données sensibles comme les clés d'API et identifiants de base de données

☐ C. À définir l'environnement graphique de l'application

☐ D. À lister les dépendances du projet à installer

34. Qu'est-ce qu'une route ou un endpoint dans une API ?

☐ A. Une fonction de tri des données côté client

☒ B. Une URL spécifique exposée par l'API à laquelle un client peut envoyer des requêtes pour interagir avec l'application

☐ C. Le chemin d'accès réseau vers le centre de données

☐ D. Le fichier de configuration du nom de domaine

35. Que signifie le code de statut HTTP 404 ?

☐ A. Le serveur a rencontré une erreur interne inconnue

☒ B. La ressource demandée n'a pas été trouvée sur le serveur

☐ C. La requête a été traitée avec succès

☐ D. L'utilisateur n'est pas authentifié pour accéder à cette ressource

36. Comment faire communiquer un Frontend et un Backend de manière sécurisée et s'assurer que les données arrivent correctement ?

☐ A. En écrivant les données directement dans la base de données depuis le client web

☐ B. En utilisant uniquement des cookies partagés sans API

☒ C. En utilisant des requêtes HTTP (ex: fetch, axios) vers des endpoints de l'API avec des validations de schéma des deux côtés

☐ D. Le frontend et le backend ne peuvent pas communiquer directement au runtime

37. Quelle est l'interprétation correcte des codes de statut HTTP suivants : 200, 404, et 500 ?

☒ A. 200 = Requête réussie (OK) ; 404 = Ressource non trouvée ; 500 = Erreur interne du serveur

☐ B. Ce sont des codes d'erreur de base de données uniquement

☐ C. 200 = Ressource créée ; 404 = Accès interdit ; 500 = Succès

☐ D. 200 = Erreur de syntaxe ; 404 = Redirection ; 500 = Version non supportée

38. Dans un scénario de coding avec un assistant IA (comme Claude), comment un développeur doit-il s'y prendre pour coder de manière productive et propre ?

☒ A. Fournir un contexte précis, procéder par étapes itératives, et toujours relire, tester et valider le code généré avant de l'intégrer

☐ B. Faire une confiance absolue au code généré par l'IA sans jamais faire de tests locaux

☐ C. Copier-coller l'intégralité du projet sans contexte et lui demander de tout réécrire d'un coup

☐ D. Ne lui confier que la rédaction des commentaires

39. [BONUS] Comment peut-on personnaliser ou créer un agent/comportement spécifique avec Claude Code ou des assistants IA similaires ?

☐ A. En écrivant le code uniquement en langage assembleur

☐ B. En modifiant directement le code source binaire de l'IA

☐ C. En payant un abonnement développeur obligatoire auprès du fournisseur cloud

☒ D. En définissant des règles, des instructions personnalisées (custom instructions/system prompts) ou des compétences (skills) dans les répertoires de configuration

40. Qu'est-ce que le 'Vibe Coding' et quelle est la meilleure pratique pour éviter que le code ne devienne instable ?

☐ A. Laisser l'IA générer tout le code dans un seul gros fichier sans architecture

☐ B. Supprimer tous les linters et outils de type checking pour coder plus vite

☐ C. Coder en écoutant de la musique sans se soucier des bugs

☒ D. Programmer de manière fluide assistée par l'IA en se concentrant sur la logique de haut niveau tout en maintenant une structure modulaire, des tests automatisés et des commits réguliers

41. Quelles sont les meilleures pratiques pour optimiser le code source et rendre une application web/mobile fluide pour l'utilisateur ?

☐ A. Multiplier les boucles imbriquées complexes et les requêtes synchrones bloquantes

☐ B. Utiliser des images haute résolution non compressées et charger toutes les données au démarrage

☐ C. Désactiver le garbage collector et utiliser uniquement des variables globales

☒ D. Implémenter le lazy loading, la pagination des données, la mise en cache, et optimiser les composants pour éviter les rendus inutiles

42. Dans les architectures frontend modernes, à quoi sert un gestionnaire d'état global (comme Redux, Zustand ou Pinia) ?

☐ A. À remplacer le serveur backend pour les requêtes de base de données

☒ B. À centraliser et synchroniser l'état partagé entre des composants éloignés dans l'arbre d'affichage

☐ C. À stocker les mots de passe des utilisateurs en clair

☐ D. À optimiser la vitesse de chargement des images

43. Quelle est la principale différence entre une API REST classique et GraphQL ou WebSockets ?

☐ A. GraphQL est plus lent que REST dans toutes les situations possibles

☐ B. WebSockets ne fonctionne que sur les terminaux Android

☐ C. REST est le seul protocole qui supporte le format JSON

☒ D. REST utilise des endpoints fixes pour chaque ressource ; GraphQL permet de requêter précisément les données requises ; WebSockets permet une communication bidirectionnelle en temps réel

44. Dans un environnement asynchrone mono-thread (comme JavaScript/Node.js), comment sont traitées les opérations lourdes pour éviter de bloquer l'interface ?

☐ A. En bloquant tout le rendu visuel jusqu'à la fin de l'opération

☒ B. En déléguant les opérations I/O ou asynchrones à l'Event Loop et en utilisant des promesses (Promises) ou des Web Workers

☐ C. En forçant le processeur à s'arrêter temporairement

☐ D. Le mono-thread ne permet pas de gérer l'asynchronisme

45. Qu'est-ce que le CORS (Cross-Origin Resource Sharing) et pourquoi est-il important ?

☐ A. Un framework de style CSS alternatif à Tailwind

☐ B. Un protocole de compression de base de données

☐ C. Un système pour partager les sessions utilisateurs entre différents serveurs physiques

☒ D. Un mécanisme de sécurité du navigateur qui restreint les requêtes HTTP d'un site web vers un domaine différent de celui qui a servi la page

Cas Pratiques

46. Un utilisateur est débité deux fois après avoir cliqué deux fois sur « Payer ». Quelle solution proposez-vous côté client et côté serveur ?

Éléments de réponse attendus :

- Idempotency key côté API (unique par tentative de transaction).
- Désactiver le bouton après clic (frontend) + debounce.
- Vérification côté serveur du statut de la commande avant traitement.
- Transaction atomique + verrou (lock) sur la ressource concernée.

47. Votre écran d'accueil met plus de 10 secondes à charger. Comment améliorer les performances ?

Éléments de réponse attendus :

- Profiler l'app pour identifier le goulot d'étranglement (réseau, rendu, calculs).
- Lazy loading / pagination des données.
- Mise en cache locale + skeleton screens pendant le chargement.
- Réduire la taille des images/assets (compression, formats optimisés).
- Optimiser les appels API (parallélisation, réduction du payload).

48. Une API devient lente avec beaucoup d'utilisateurs. Que mettez-vous en place ?

Éléments de réponse attendus :

- Mise en cache (Redis/CDN).
- Load balancing et scaling horizontal.
- Optimisation des requêtes DB (index, pagination).
- Rate limiting / throttling.

- Architecture asynchrone avec queues pour les tâches lourdes.

49. Votre API produit des centaines de requêtes SQL pour charger une liste d'éléments avec leurs relations (problème des requêtes N+1). Comment l'optimiser ?

Éléments de réponse attendus :

- Utiliser des jointures SQL ou le préchargement des relations (eager loading) pour limiter les requêtes.
- Utiliser des outils de debug de requêtes SQL pour identifier les N+1.
- Mise en cache (Redis) des résultats fréquents.
- Optimiser les serializers/DTOs pour éviter les accès relationnels non maîtrisés.

50. Vous devez déployer une nouvelle version sans interruption de service. Quelle stratégie utilisez-vous (blue-green / canary) ?

Éléments de réponse attendus :

- Blue-Green : deux environnements identiques, bascule instantanée du trafic via load balancer.
- Canary : déploiement progressif à un petit pourcentage d'utilisateurs, monitoring puis extension progressive.
- Rollback rapide en cas d'anomalie détectée.
- Health checks automatisés avant bascule complète.